

Mecanismos de segurança: CORS, autenticação e autorização

Autenticação e Autorização

- Autenticação: login retorna token
- Autorização: filtro verifica token
- Uso de JWT no NodeJS
npm i **jsonwebtoken**
- Rota de login para verificação de senha
- Classe para gerência de tokens

```
const jwt = require('jsonwebtoken')
const secret = 'RX#hl$bX5sbw%9xW';
```

```
class TokenManager {
```

```
  verifyJWT = (req, res, next) => {
    const token = req.headers['x-access-token'];
    if (!token) return res.status(401).json(
      { auth: false, message: 'No token provided.' });
```

```

jwt.verify(token, secret, (err, decoded) => {
  if (err) return res.status(500).json(
    { auth: false,
      message: 'Failed to authenticate token.' });
  req.userId = decoded.id;
  next(); // filtro: acrescenta userId e segue para o próximo
});
}

```

```
sign = (id) => {
  return jwt.sign({ id }, secret, { expiresIn: 300 });
  // token expira em 5 min
}

module.exports = TokenManager
```

```
router.post('/login', async (req, res) => {
  const usuario = await dao.findByLoginSenha(
    req.body.login, req.body.senha);
  if(usuario){
    const id = usuario.id; const token = tm.sign(id);
    return res.json({ auth: true, token: token });
  }
  res.status(500).json({message: 'Login inválido!'});
})
```

```
router.post('/addUser', tm.verifyJWT, async(req,res)=>{
  const usuario = await dao.create(req.body);
  return res.json({usuario: usuario});
})
```

POST		http://localhost:3000/auth/addUser		SEND	
JSON	HEADERS 1	QUERY STRING	AUTH	ASSERTION	BODY
☒ x-access-token		3M_NzrRgUgcEF3QPRCP4jmk		200 OK	
				1 { 2 "usuario": { 3 "id": 9, 4 "nome": "Novo Usuario", 5 "login": "novo7", 6 "senha": "\$2a\$10\$0e3HMHUg06IPIImn0jXl1jauwYMFIE	